Behavioral Runtime Governance for Autonomous AI Agents: A Systematic Comparison

Anonymous Authors

Abstract

Most current approaches to AI agent safety apply constraints at the level of individual decisions — fixed allow/deny lists, per-call classifiers, or declarative policy rules — without considering how an agent's behavioral state evolves over time. We present a behavioral runtime governance model that dynamically adjusts an agent's permitted action set based on accumulated behavioral state: mode activation patterns, stress level, energy budget, and behavioral drift. The model defines four stress levels (LOW through OVER) that progressively tighten action-space constraints through a deterministic pipeline, with graduated restriction at HIGH and hard lockdown at OVER. We evaluate this model in a systematic comparison against five baselines representing different points in the guardrails design space: no guardrails, static filtering, threshold-based circuit breaking, per-action classification, and declarative policy engines. Across three metric categories (safety, utility, adaptivity) and four scenarios totaling 7,200 decision cycles (4 scenarios × 300 turns × 6 systems), the behavioral runtime's primary advantage appears in adaptivity (0.590 vs 0.201 for the nearest baseline) and temporal state awareness, at a clear utility cost (0.413, the lowest in the comparison). Extended 1000-turn evaluation across four workloads shows that risky action exposure drops 78–84% compared to a prior-generation engine. Among the tested systems, the behavioral runtime is the only one that maintains the temporal state necessary for drift detection (90.5% detection rate vs 0% for the other baselines), energy tracking, and stability monitoring. These results suggest that endogenous, behavior-state-adaptive governance — restricting what is permitted based on runtime context, not only what is categorically forbidden — is a complementary layer worth exploring in the agent safety stack.

1. Introduction

The deployment of autonomous AI agents — systems that take multi-step actions in external environments based on language model reasoning — has created an urgent need for runtime behavioral governance. Current safety approaches fall into two broad categories with a shared limitation.

The first is prompt-level safety: constitutional principles (Bai et al., 2022), RLHF training (Ouyang et al., 2022), or system instructions that shape behavior before deployment. These operate at the token generation level and provide no runtime guarantees once the model is deployed as an agent. The second is framework-level middleware: runtime systems that intercept agent actions and apply constraints. These range from static allow/deny lists (Guardrails AI, 2023) to input/output validators (Rebedea et al., 2023), safety classifiers (Inan et al., 2023), and declarative policy engines (Invariant Labs, 2024; Galileo, 2024).

Both categories share a fundamental limitation: they treat each decision cycle independently. A static filter applies the same permissions whether the agent is in its first minute of operation or its thousandth, whether the environment is calm or under sustained adversarial pressure. A per-call classifier evaluates each action in isolation without awareness of how the agent's behavioral state has evolved over time. Even sophisticated policy engines with rate limits and cost budgets track only shallow state (counters and timers) rather than the agent's behavioral trajectory.

This paper presents a behavioral runtime governance model that maintains deep state across decision cycles and uses that state to dynamically govern the agent's action space. The core insight is that **the set of actions an agent should be permitted to take is not fixed, but should vary based on the agent's accumulated behavioral state** — its current stress level, the modes of behavior that are active, its remaining energy budget, and whether its behavior has drifted from its established baseline.

We make three contributions:

1. A formal stress-adaptive governance model with four stress levels, graduated action restriction at HIGH (removing high-risk actions) and hard lockdown at OVER (restricting to 4 safe actions), with deterministic causal traceability from observation to contract output.

2. A systematic comparison framework evaluating six guardrail paradigms (B0–B5) across three metric categories — safety, utility, and adaptivity — using four scenarios and 7,200 decision cycles at the 300-turn scale (4 scenarios × 300

turns × 6 systems) plus 8,000 cycles at the 1000-turn scale (4 workloads × 1,000 turns × 2 engine versions).

3. Empirical evidence that behavioral runtime governance provides a distinctive advantage in adaptivity (drift detection, energy tracking, stability monitoring) — capabilities that fall outside the architectural scope of the other tested paradigms — while incurring a measurable utility cost from mode-driven permission narrowing.

2. Related Work

Static guardrails. Guardrails AI (2023) provides schema validation for LLM outputs. NeMo Guardrails (Rebedea et al., 2023) uses a dialog management approach with programmable rails. These systems are designed around per-request rule enforcement — a different design point from session-level behavioral tracking.

Safety classifiers. LlamaGuard (Inan et al., 2023) and LlamaFirewall (Meta, 2025) classify inputs and outputs as safe or unsafe using fine-tuned language models. These achieve high accuracy on individual decisions by operating per-call. Their design optimizes for single-turn precision rather than multi-turn behavioral pattern detection — a complementary rather than competing capability.

Policy engines. Invariant Labs (2024) and Galileo Agent Control (2024) offer declarative policy languages with rate limits, cost budgets, and rule composition. These systems maintain operational state (counters, timers, budgets) and are well-suited to enforcement of explicit constraints. Our work explores a different axis: tracking emergent behavioral trajectories (mode activation, drift, energy) that are not easily expressed as declarative rules.

Cognitive architectures. ACT-R (Anderson et al., 2004) and SOAR (Laird, 2012) model stress and cognitive load in human cognition simulations. Recent work on cognitive architectures for LLM agents (Sumers et al., 2023) explores similar patterns. Our model draws on this tradition but applies it specifically to runtime action governance rather than cognitive modeling.

Agent safety research. Surveys of LLM agent risks (Ruan et al., 2024; Xi et al., 2023) identify action safety as a key concern, focusing on risk taxonomies. AgentBench (Liu et al., 2023) evaluates agent capabilities. Our work complements these by proposing a runtime mitigation mechanism rather than a risk framework or capability benchmark.

Existing systems largely emphasize event-local moderation or externally specified rule enforcement. Our work explores a different axis — using endogenous behavioral state (accumulated mode activation, stress trajectory, energy budget, and drift detection) as the basis for runtime governance that dynamically adjusts the permitted action set. We position this as a complementary layer to existing per-call and per-rule approaches, not a replacement.

3. Behavioral Runtime Governance Model

3.1 Overview

The governance model operates as middleware between the agent framework (e.g., LangGraph, CrewAI) and the language model. At each decision cycle, the agent sends an observation to the middleware, which processes it through a deterministic pipeline and returns an **execution contract** — a read-only specification of permitted actions, current stress level, energy budget, and behavioral metadata.

The pipeline consists of seven stages: (1) mode activation, (2) energy update, (3) stress and state evaluation, (4) arbitration, (5) composite detection, (6) drift detection, and (7) stability computation. This paper focuses on stages 1, 3, 6, and the contract output.

3.2 Mode Activation

The system maintains a vector of seven behavioral modes. Each observation targets a specific mode with a signal strength s \in [-1, 1] and confidence c \in [0, 1]. Mode values are updated via exponential moving average:

```
v_{t+1}(m) = v_t(m) \cdot (1 - \alpha \cdot c) + s \cdot \alpha \cdot c
```

where \alpha is the base learning rate. Only the targeted mode updates per cycle. One mode is designated as the **stress response** mode; distress signals (blocked actions, errors, resource contention) target this mode.

A stress-specific learning rate multiplier accelerates stress response mode updates, enabling faster detection of environmental pressure without affecting other modes:

```
\alpha_{\text{stress}} = \alpha \cdot \beta, \quad \beta > 1
```

3.3 Stress Classification and Level-Dependent Bleed

The stress response mode's absolute value determines stress level via threshold comparison:

$$\text{stress\_level}(t) = \begin{cases} \text{OVER} & \text{if } |v_t(\text{stress})| \geq \theta_{\text{over}} \\ \text{HIGH} & \text{if } |v_t(\text{stress})| \geq \theta_{\text{high}} \\ \text{MED} & \text{if } |v_t(\text{stress})| \geq \theta_{\text{med}} \\ \text{LOW} & \text{otherwise} \end{cases}$$

Because the stress response mode updates via EMA, stress level changes smoothly — sustained pressure is required for escalation. A key design element is **level-dependent bleed**: the rate at which stress decays varies by level. At HIGH, the bleed rate is halved ($r_{\text{HIGH}} = 0.01$), preventing rapid oscillation between restricted and unrestricted states. At OVER, bleed is conditional on input characteristics — calm signals allow faster recovery, while continued pressure maintains the lockdown. This eliminates the "oscillation vulnerability" where an attacker can exploit brief windows of unrestricted access between stress transitions.

3.4 Graduated Action-Space Governance

The execution contract's permitted action set is governed by a two-tier mechanism.

Mode-driven permissions (normal operation). Each active mode (strength above threshold) contributes a subset of actions from a vocabulary of $|A| = 12$ standard actions. The permitted set is the union of all active modes' contributions. Under mixed-operation conditions with diverse inputs, several active modes may yield a broader vocabulary (up to 10–12 actions). However, steady-state scenarios with repetitive input patterns may activate only 2 modes, yielding as few as 5 available actions — a tradeoff we examine in Section 5.1.3.

Stress-driven restriction (elevated state). At HIGH stress, four high-risk actions (proactive behaviors: initiating exploration, challenging decisions, executing plans, changing direction) are removed from the permitted set, regardless of mode activity. At OVER, the permitted set collapses to a fixed safe set of 4 reactive and disengagement actions:

$$A_{\text{HIGH}} = A_{\text{mode}} \setminus A_{\text{risky}}, \quad |A_{\text{risky}}| = 4$$

$$A_{\text{OVER}} = \{a_1, a_2, a_3, a_4\} \subset A_{\text{reactive}} \cup A_{\text{disengage}}$$

Two additional actions are **permanently forbidden** regardless of stress level, functioning identically to a static guardrail. The graduated model governs the space between "always forbidden" and "always allowed."

The restriction mechanism satisfies two formal properties: (1) **monotonicity** — $A_{\text{OVER}} \subset A_{\text{HIGH}} \subseteq A_{\text{mode}}$, i.e., higher stress levels never expand the action set; (2) **subset containment** — the OVER safe set is always a subset of any HIGH-restricted set, ensuring no discontinuous jumps in permitted actions across stress transitions.

3.5 Drift Detection

The system detects behavioral drift by comparing the current mode activation vector against an established anchor. The anchor is initialized after a warm-up period (20 cycles) and refreshes only during stable periods (drift magnitude below D_0 threshold). Magnitude is computed via L2 norm across the mode vector, yielding four drift levels: D_0 (noise), D_1 (context drift), D_2 (adaptive drift), D_3 (corrupt drift with auto-rollback).

In the current implementation, drift detection serves primarily as a **monitoring and alerting signal**. D1 and D2 events are surfaced to operators but do not directly modify the action space. Only D3 triggers an automatic rollback to the established anchor. This design reflects a deliberate choice: drift detection identifies *when* behavior has shifted, leaving the response policy to the operator. Future work may explore D2-driven action narrowing as an optional configuration.

3.6 Energy Model

Energy is tracked as a continuous variable in [0, 1], consumed by active modes and recovered at a rate determined by system state. An energy floor of 0.20 prevents depletion during non-stress operation. Under HIGH/OVER stress, the floor is bypassed, preserving energy depletion's significance as a genuine distress signal.

3.7 Deterministic Traceability

The entire pipeline is deterministic and contains no LLM inference. Given an observation sequence, any contract output is exactly reproducible. This is critical for production auditability.

4. Systematic Comparison Framework

4.1 Baselines

We compare the behavioral runtime against five baselines representing different points in the guardrails design space. Each baseline is a representative mechanism-level simulation inspired by published system descriptions, not a claim of exact product reimplementation. The intent is to compare *architectural paradigms* at a controlled abstraction level:

Baseline	Paradigm	State Depth	Description
B0	No guardrails	0	All actions always available, including forbidden
B1	Static filter	0	Fixed allow/deny list (Guardrails AI-style)
B2	Circuit breaker	1	N consecutive anomalies trigger safe mode (rate limiter)
B3	Per-action classifier	1	Binary safe/unsafe per action (LlamaGuard-style, TPR=0.90, FPR=0.08)
B4	Policy engine	3	Declarative rules + rate limits + cost budgets (Invariant-style)
B5	Behavioral runtime	7	Full model: modes + stress + energy + drift + stability

State depth counts the number of independent state dimensions tracked across decision cycles.

4.2 Metrics

We evaluate across three categories:

Safety. (1) Forbidden action blocking rate; (2) risky action exposure during stress turns (proportion of high-risk actions permitted when ground-truth indicates stress); (3) escalation containment (how quickly the system restricts after stress onset).

Utility. (1) Average action vocabulary size; (2) non-stress restriction rate (restriction during non-stress turns); (3) recovery score (how quickly full vocabulary is restored after stress ends).

Adaptivity. (1) Proportionality (correlation between input intensity and restriction); (2) state awareness depth; (3) drift detection rate; (4) energy and stability tracking.

Composite scores weight safety (40%), utility (30%), and adaptivity (30%).

4.3 Scenarios

Four scenarios at 300 turns each, evaluated across all six systems (7,200 total decision cycles):

Steady State (A). Consistent alternating input, no perturbation. Tests false-activation and baseline utility.

Gradual Shift (B). Behavioral signals transition between modes over 300 turns. Tests drift detection.

Stress Spike (C). Sudden high-intensity stress signals at turns 100–149, recovery at 150+. Tests escalation, containment, and recovery.

Chaotic (D). Random modes, strengths, and confidence values. Tests resilience under sustained disorder.

4.4 Extended Evaluation

We additionally run four realistic 1000-turn workloads comparing the current engine against a prior-generation engine (v1, with uncalibrated parameters):

S1 Enterprise Agent. Customer service: onboarding (0–200), routine (200–400), adversarial attack (400–549), recovery (550+).

S2 Autonomous Researcher. Exploration, focus shift, intermittent stress, long-tail operation.

S3 Adversarial Marathon. 900+ turns of sustained adversarial input after brief warm-up.

S4 Multi-Mode Chaos. Sinusoidal stress probability with random mode mixing.

5. Results

5.1 Systematic Comparison (300-turn, 6 baselines)

We first present per-dimension results before combining them into composite scores, so that readers can evaluate each dimension independently.

5.1.1 Safety Breakdown: Stress Spike Scenario (C)

Metric	B0	B1	B2	B3	B4	B5
Forbidden block rate	0%	100%	100%	100%	100%	100%
Risky exposure (stress turns)	100%	100%	20%	9%	20%	18%
Escalation containment	0.00	0.00	0.92	1.00	0.96	1.00

B0 and B1 have no stress awareness — risky actions remain fully available during stress. B2's circuit breaker activates after 5 consecutive anomalies (containment = 0.92). B3's per-action classifier achieves the lowest per-turn exposure (9%) through independent per-action evaluation. B5's escalation containment score (1.00) measures how quickly the system *enters a restricted state* after stress onset — within 1 cycle, the EMA accumulates past the HIGH threshold. The 18% residual risky exposure reflects the fact that HIGH restriction removes 4 of 12 actions, not all of them; full lockdown requires reaching OVER. The two metrics measure different properties: containment measures response latency, while risky exposure measures restriction severity.

5.1.2 Drift Detection

System	B gradual (detection rate)	D chaotic (detection rate)
B0–B4 (all baselines)	0.0%	0.0%
B5 BehavioralRuntime	**90.5%**	**88.7%**

None of the B0–B4 baselines are designed to detect behavioral drift — this capability requires maintaining temporal state (mode activation trajectories and anchor-based deviation measurement) that falls outside their architectural scope.

5.1.3 Utility Tradeoff

B5's utility composite (0.413) is the lowest in our comparison, driven by mode-driven permission narrowing in steady-state scenarios (average 5.0 allowed actions vs B1's 12.0). This metric reflects nominal mode-conditioned narrowing — the system restricts actions based on which behavioral modes are active — rather than erroneous stress-triggered lockdown. Under steady-state input, typically 2 modes are active, yielding a narrower vocabulary by design. We discuss the implications and potential mitigations in Section 6.

5.1.4 Composite Scores

System	Safety	Utility	Adaptivity	Overall
B0 NoGuardrails	0.300	0.975	0.000	0.413
B1 StaticFilter	0.700	0.925	0.000	0.558
B2 ThresholdGating	0.829	0.918	0.092	0.635
B3 ClassifierGuard	0.935	0.651	0.201	0.629
B4 PolicyDSL	0.952	0.604	0.094	0.590
B5 BehavioralRuntime	**0.905**	**0.413**	**0.590**	**0.663**

Under our weighting scheme (Safety 40%, Utility 30%, Adaptivity 30%), the behavioral runtime achieves the highest composite score. This result is sensitive to the chosen weights — B5's advantage is driven primarily by adaptivity (0.590 vs 0.201), which compensates for its utility deficit. Readers who weight utility more heavily will reach different conclusions; we provide the per-dimension scores above to support independent evaluation.

5.2 Extended Evaluation (1000-turn, v1 vs current engine)

The 1000-turn evaluation compares the prior-generation engine (v1: uncalibrated energy, no drift detection, no graduated restriction) against the current engine across realistic workloads.

5.2.1 Safety: Risky Action Exposure

Scenario	v1 Risky Exposure	Current Risky Exposure	Reduction
S1 Enterprise Agent	10%	2%	78%
S2 Autonomous Researcher	3%	1%	50%
S3 Adversarial Marathon	0%	0%	(both zero)
S4 Multi-Mode Chaos	22%	3%	84%

The largest improvement occurs in S4 (22% to 3%, 7x reduction). The current engine's graduated restriction (HIGH removes risky actions before OVER) eliminates the gaps that v1's single-tier OVER lockdown leaves during stress oscillation.

In S1, the current engine's stress response during the adversarial attack phase (turns 400–549) shows 4 entries/exits vs v1's 10 entries/exits. Fewer transitions mean fewer windows of unrestricted access — this is the mechanism behind the 78% reduction.

5.2.2 Signal Quality: Energy and Drift

Scenario	v1 Energy Critical Turns	Current Energy Critical	v1 D1+ Events	Current D1+ Events
S1 Enterprise	821/1000 (82%)	263/1000 (26%)	952 (95%)	277 (28%)
S2 Researcher	724/1000 (72%)	135/1000 (14%)	936 (94%)	678 (68%)
S3 Adversarial	738/1000 (74%)	472/1000 (47%)	746 (75%)	300 (30%)
S4 Chaos	958/1000 (96%)	721/1000 (72%)	950 (95%)	778 (78%)

v1's energy depletes to critical in 72–96% of turns, rendering it an uninformative signal. The current engine's energy floor eliminates false depletion during normal operation; critical turns now concentrate in genuine stress periods.

v1's drift detection flags 75–95% of turns as D1+, creating alert fatigue. The current engine's calibrated thresholds and delayed anchor reduce D1+ to 28–78%, with detections concentrated at genuine behavioral phase boundaries.

5.2.3 Behavioral Stability

Scenario	v1 Final Stability	Current Final Stability	Improvement
S1 Enterprise (1000 turns)	0.59	0.87	+47%
S3 Adversarial (1000 turns)	0.20	0.47	+135%

After 900+ turns of sustained adversarial input (S3), v1 stability drops to 0.20, while the current engine holds at 0.47. This suggests improved resilience for long-duration sessions, though the practical threshold for "acceptable stability" remains application-dependent.

5.2.4 Summary

The 1000-turn results point to three practical improvements from the current engine: (1) **safer under stress** — graduated restriction reduces risky exposure 78–84% without full lockdown; (2) **more meaningful signals** — energy and drift detections concentrate in genuine stress/shift periods rather than firing continuously; (3) **more stable long-term** — behavioral stability improves 47–135% across tested workloads.

6. Discussion

6.1 The Adaptivity Dimension

In our comparison, B0–B4 score 0.000–0.201 on adaptivity, while B5 scores 0.590. This gap reflects a difference in architectural scope rather than implementation quality. B0–B2 track zero or one state dimension; B3 evaluates each action independently; B4 maintains counters and budgets (3 dimensions). B5 tracks mode activation trajectories, drift anchors, energy budgets, stress history, and stability indices (7+ dimensions).

Drift detection illustrates this difference: 90.5% detection rate for B5 vs 0% for all baselines in our evaluation. The baselines are not designed to detect behavioral drift — they do not maintain the temporal state that would make drift a meaningful concept within their frameworks. Whether this additional capability justifies the added complexity is an open question that depends on the deployment context.

6.2 The Utility Cost

B5's utility composite (0.413) is the lowest in the comparison. This is the cost of mode-driven governance: by restricting actions to those contextually appropriate for the agent's current behavioral mode, the system narrows the action vocabulary even in non-stress conditions.

We view this as a design tradeoff, not a defect. Mode-driven narrowing encodes the principle that an agent in perception mode should not impulsively execute, just as an agent under stress should not explore. However, we acknowledge that production deployments may require configurable permissiveness — a "strict / relaxed / stress-only" mode selector that lets operators choose their preferred point on the safety-utility curve.

6.3 Graduated vs. Binary Restriction

The move from v1's single-tier restriction (OVER only) to the current engine's graduated restriction (HIGH removes 4 risky actions, OVER hard-locks 4 safe actions) is the change with the largest practical impact. In the 1000-turn S4 chaos scenario, risky exposure drops from 22% to 3% — almost entirely attributable to HIGH-level restriction engaging earlier and more frequently than OVER alone.

The graduated model also produces fewer stress oscillations (S1: 4 transitions vs 10), which means fewer exploitable windows and more predictable behavior for downstream agent logic.

6.4 Limitations

No user study. We demonstrate governance behavior in synthetic scenarios but do not measure downstream task performance. Whether stress-governed agents produce better outcomes for end users remains an open question.

Simulated baselines. B3 (classifier) and B4 (policy engine) are mechanism-level simulations inspired by published system descriptions, not exact reimplementations of commercial products. Real-world systems with production-tuned parameters may achieve different performance characteristics. Our comparison evaluates *paradigms*, not specific products.

Utility cost. The non-stress restriction rate of 99.3% in steady-state scenarios reflects the structural cost of mode-driven governance and will require configurable permissiveness for production adoption.

Energy under sustained stress. In S4 chaos, 72% of turns remain at critical energy. The energy model's stress-period behavior requires further calibration.

7. Conclusion

We have presented a behavioral runtime governance model for autonomous AI agents and evaluated it against five baselines representing different points in the guardrails design space. The model's primary advantage appears in adaptivity — drift detection, energy tracking, and stability monitoring capabilities that fall outside the architectural scope of the tested baselines — while its principal weakness is the utility cost of mode-driven governance.

The 1000-turn extended evaluation suggests practical benefits in the tested workloads: risky action exposure drops 78–84% compared to a prior-generation engine, graduated restriction preserves a substantial portion of the action vocabulary at HIGH (removing 4 of 12 actions rather than collapsing to 4), and behavioral stability improves 47–135% in long-duration sessions. The graduated restriction mechanism (HIGH + OVER) reduces stress oscillation compared to single-tier designs, narrowing exploitable windows from 10 to 4 transitions in the enterprise workload scenario.

The model's principal limitation is the utility cost of mode-driven governance, which narrows the action vocabulary even under non-stress conditions. We view this as a design tradeoff inherent to the approach, but acknowledge that configurable permissiveness will be necessary for production adoption.

We plan to release the governance model as part of an open-source behavioral middleware system and will make evaluation artifacts available upon publication. We invite the community to evaluate the approach's applicability to their agent safety requirements.

References

Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., & Qin, Y. (2004). An integrated theory of the mind. *Psychological Review*, 111(4), 1036–1060.

Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*.

Galileo. (2024). Galileo Agent Control: Runtime safety for AI agents. https://www.galileo.ai

Guardrails AI. (2023). Guardrails: Adding guardrails to large language models. https://github.com/guardrails-ai/guardrails

Inan, H., Upasani, K., Chi, J., et al. (2023). Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*.

Invariant Labs. (2024). Invariant Analyzer: Policy-based runtime safety for AI agents. https://invariantlabs.ai

Laird, J. E. (2012). *The Soar Cognitive Architecture*. MIT Press.

Liu, X., Yu, H., Zhang, H., et al. (2023). AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*.

Meta. (2025). LlamaFirewall: An open-source guardrail system for building secure AI agents. *arXiv preprint*.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35.

Perez, E., Ringer, S., Lukosiute, K., et al. (2022). Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*.

Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., & Cohen, J. (2023). NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. *arXiv preprint arXiv:2310.10501*.

Ruan, Y., Dong, H., Wang, A., et al. (2024). Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*.

Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2023). Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*.

Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How does LLM safety training fail? *arXiv preprint arXiv:2307.02483*.

Xi, Z., Chen, W., Guo, X., et al. (2023). The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*.

Appendix A: Action Taxonomy

The system defines a vocabulary of |A| = 12 standard actions spanning proactive behaviors (initiating exploration, proposing changes, challenging decisions, executing plans), reactive behaviors (responding to queries, maintaining stability, seeking clarification), and disengagement behaviors (deferring decisions, withdrawing from activity). Two additional actions are permanently forbidden regardless of stress level.

Under HIGH stress, 4 proactive actions are removed. Under OVER stress, only 4 reactive/disengagement actions remain. This creates a three-tier governance gradient:

Stress Level	Available Actions	Restriction Mechanism
LOW / MED	3–12 (mode-dependent)	Mode-driven permission
HIGH	mode-set minus 4 risky	Stress-driven removal
OVER	4 fixed safe actions	Hard lockdown

Appendix B: Composite Score Methodology

Composite scores combine three equally important but differently weighted metric categories:

Safety (weight 0.40): 0.4 \cdot \text{forbidden_block} + 0.3 \cdot (1 - \text{risky_exposure}) + 0.3 \cdot \text{containment}

Utility (weight 0.30): 0.3 \cdot (\text{avg_actions} / 12) + 0.4 \cdot (1 - \text{non_stress_restriction}) + 0.3 \cdot \text{recovery}

Adaptivity (weight 0.30): 0.3 \cdot \text{proportionality} + 0.2 \cdot \text{arch_capability} + 0.3 \cdot \text{drift_rate} + 0.1 \cdot \text{energy} + 0.1 \cdot \text{stability}

Note: \text{arch_capability} = \min(\text{state_dims}/7, 1) is an architectural capability proxy that credits systems for maintaining more state dimensions. We include it to capture the design-level distinction between stateless and stateful governance, but acknowledge it favors architecturally richer systems by construction. Readers may wish to evaluate the per-dimension results (Section 5.1.1–5.1.3) independently of this composite.

Overall: 0.4 \cdot S + 0.3 \cdot U + 0.3 \cdot A

Scores are averaged across all four scenarios before weighting.

Appendix C: Baseline Implementation Details

B2 ThresholdGating. Signal strength \geq 0.75 counts as anomaly. Five consecutive anomalies trigger safe mode (4 actions) for 20 turns, then auto-reset.

B3 ClassifierGuard. Per-action binary classifier with TPR=0.90 (correctly blocks risky action under stress) and FPR=0.08 (incorrectly blocks safe action under no stress). Uses seeded RNG for reproducibility. Safe action floor guarantees minimum 4 actions.

B4 PolicyDSL. Three mechanisms: (1) rate limit of 10 risky actions per 50-turn window; (2) cost budget of 15.0 per 100 turns with per-action costs from 0.0–2.0; (3) consecutive stress trigger (3 high signals) with 10-turn cooldown. Safe action floor guarantees minimum 4 actions.

All baselines use identical scenario inputs, random seeds, and simulated agent behavior (uniform random action selection from all 14 actions) for fair comparison. Results are intended to compare mechanism classes under a controlled abstraction, not to claim superiority over any specific deployed product. Production systems with tuned parameters and richer integrations may perform differently.